

Exercise #12

11. April 2023

Exercises marked with a (J) should be handed in as a Jupyter notebook.

Problem 1. (Numerical solution of ODEs)

(J) In this problem we will implement Euler's method, second order Taylor's method, and Heun's method, and use them to approximate the solution to the ODE,

$$y' = (1 - 2t)y, \quad y(0) = 1.$$

The exact solution to this equation is $y(t) = e^{t-t^2}$.

- a) Implement Euler's method, and compute an approximation of $y(1)$, using a step size equal to 0.5.
- b) Do the same using Heun's method and the second order Taylor method. The second order Taylor method is given by

$$y_{n+1} = y_n + hf(t_n, y_n) + \frac{h^2}{2}f'(t_n, y_n),$$

where $f(t, y) = y'(t)$ and thus $f'(t, y) := \frac{\partial f}{\partial t}(t, y) = y''(t)$.

- c) We now want to approximate the convergence orders of these methods numerically. Recall that we defined the global error,

$$\epsilon_g := \max_i |y(t_i) - y_i|.$$

If we assume that $\epsilon_g(h) \approx Mh^p$, for some $M > 0$, we have,

$$\log \left(\frac{\epsilon_g(h_1)}{\epsilon_g(h_2)} \right) \approx p \log \left(\frac{h_1}{h_2} \right).$$

Compute the global error of the methods from a)-c) using $h_1 = 10^{-2}$ and $h_2 = 10^{-3}$, where $t_i = ih, i = 1, \dots, \frac{1}{h}$. Use this to approximate the convergence order, p , for each of the three methods.

- d) We can also approximate the convergence order by plotting $\log(\epsilon_g(h)) = \log(M) + p \log(h)$ versus $\log(h)$, and inspecting the slope of the function.

Plot $\log(\epsilon_g(h))$ versus $\log(h)$ for $h = 10^{-2}, 10^{-3}, 10^{-4}$ for each of the three methods.

Problem 2. (System of ODEs)

Let $a > 0, b > 0, c > 0, d > 0$ be constants. Write the second order linear ODE,

$$\begin{aligned} ay + by' + cy'' + d &= 0, \\ y(0) &= -1, \\ y'(0) &= 1, \end{aligned}$$

as a linear system of first order ODEs, and perform one step of Euler's method (resulting in an expression that includes the constants a, b, c, d) with step size 1.

Problem 3. (Order of a Runge–Kutta method)

In this exercise we will study a Runge–Kutta method that is given by

$$k_1 = y_n + hf(t_n, y_n) \quad (1)$$

$$k_2 = y_n + hf\left(t_n + \frac{h}{3}, y_n + \frac{k_1}{3}\right) \quad (2)$$

$$k_3 = y_n + hf\left(t_n + h\frac{2}{3}, y_n + \frac{2}{3}k_2\right) \quad (3)$$

$$y_{n+1} = y_n + \frac{h}{4}(k_1 + 3k_3) \quad (4)$$

- a) Present the method in the form of a Butcher tableau.
- b) Decide the order of the method.

Problem 4. (Lipschitz continuity)

Decide if the following functions are Lipschitz continuous for all $t, y \in \mathbb{R}$.

a) $f(t, y) = \frac{y}{t^2}$.

b) $f(t, y) = \frac{\sin(t)}{t}y$

Problem 5. (Implementation of an ODE solver)

```
import numpy as np

f = lambda t,y : 2/t**2*y
t0, tend = 1, 2
y0 = 1
N = 10

y = np.zeros(N+1)
t = np.zeros(N+1)
y[0] = y0
t[0] = a

for n in range(N):
    k1 = f(t[n],y[n])
    k2 = f(t[n]+0.5*h, y[n]+0.5*h*k1)
    y[n+1] = y[n] + h*k2

print('t=',t)
print('y=',y)
```

- There are three bugs in this code. Two that prevents it from running at all, and one which causes a completely nonsense output. Find and correct the errors.
- Which mathematical problem does this code intend to solve numerically?
- Which specific algorithm has been applied to the problem? No specific name is required, but present the method in the form of a Butcher tableau, and decide the order of the method.
- Find the first two elements of the NumPy vector y , given that point a) is accomplished.

The next exercises are optional and should not be handed in!

Problem 6. (Adaptivity)

Consider the following implementation of a Runge–Kutta method:

```
import numpy as np
import matplotlib.pyplot as plt

T = 2.0
y0 = 0.0
t = 0.0
h = 0.25

def f(t,y):
    return t*np.exp(-y)

ys = [y0]
ts = [t]

while(t+h < T):
    t, y = ts[-1], ys[-1]
    k1 = f(t,y)
    k2 = f(t+h/2, y + h*k1/2)
    k3 = f(t+3*h/4, y + 3*h*k2/4)
    k4 = f(t+h, y + h*(2*k1 + 3*k2 + 4*k3)/9)
    y = y + h*(7*k1/24 + k2/4 + k3/3 + k4/8)
    ys.append(y)
    ts.append(t + h)

plt.plot(ts, ys, 's-')
```

- Based on the implementation above, write down the Butcher tableau for this RK method.
- If we run the code, how many time steps will be computed?
- Based on the tableau, determine the order of this method.
- Considering the same initial value problem and time-step size as in Problem 1, compute the first time step using the method implemented above.
Hint: before you actually compute the stage derivatives k_i , check whether you can re-use any of the calculations done for Problem 1.
- Consider now the combination of this method with the one from Problem 1, to create

an adaptive scheme. Remember the error estimate:

$$\hat{\epsilon}_{n+1} = |y_{n+1} - y_{n+1}^*| = h \left| \sum_{i=1}^s (b_i - b_i^*) k_i \right|,$$

with the superscript * referring to the lowest-order method among the two. Based on the calculations done so far, compute $\hat{\epsilon}_1$.

- f) Comparing $\hat{\epsilon}_1$ with the tolerance $\text{tol} = 0.001$, check if the first step we computed is acceptable and, if not, compute the new time-step size h_{new} based on the formula developed in class:

$$h_{\text{new}} = P \left(\frac{\text{tol}}{\hat{\epsilon}_1} \right)^{\frac{1}{p+1}} h,$$

with $P = 0.9$.

Problem 7. (Stability)

For an ODE $y'(t) = f(t, y)$, the so-called *explicit midpoint method* is given by

$$y_{n+1} = y_n + hf(t_n + 0.5h, y_n + 0.5hf(t_n, y_n)).$$

Consider, in particular, the linear autonomous equation where $f(y) = \lambda y$, with $\lambda \in \mathbb{C}$.

- Write down the Butcher tableau for this Runge–Kutta method.
- Determine the stability function $R(z)$ of the explicit midpoint method, that is, the function that allows us to write $y_{n+1} = [R(h\lambda)]y_n$.
- Determine the stability region $\mathcal{S}$.
- For $\lambda = -20$, what is the interval of time-step sizes h for which we get a stable solution?